Turbo Streams and ActionCable

The turbo-rails gem, which we added when we installed Hotwire and Turbo in Chapter 1, *Getting Started with Client-Side Rails*, provides a set of helpers for managing ActionCable connections. Those helpers allow Rails to automatically send Turbo Stream HTML over the ActionCable connections and then enable the Turbo Stream actions to be executed on the client when the message is received.

Very broadly, these helpers allow us to do three things:

- Connect a view to an ActionCable channel via a helper method in the view itself
- Broadcast a Turbo Stream to an ActionCable channel as part of the response to a controller request
- Broadcast a Turbo Stream to an ActionCable channel automatically as a callback when an ActiveRecord is modified

The helpers provided by the turbo-rails gem used for communicating via ActionCable differ in how much default behavior they assume and how much you can specify. At one end, you can just send arbitrary content over an ActionCable channel managed by Turbo Streams. Or you can send content, and turbo-rails can wrap it in a Turbo Stream HTML structure. Or you can take advantage of the defaults and have the default partial for your model sent to a stream with a default name. Which option you choose depends on how closely your need can be modeled by Rails conventions and also how comfortable you are defining channel behavior in your ActiveRecord model.

Preparing Our Views

ActionCable is particularly useful not just for replacing polling the server, but also in allowing us to keep multiple browser sessions in sync, whether that means different devices for the same user, or multiple users looking at the same information. In this section, we're going to make it so that declaring a concert a